A Project Report

On

IMPLEMENTATION OF NEURAL NETWORK MESH GENERATION ALGORITHMS FOR 3D DOMAIN

BY

ARYAN AGARWAL

2024A7PS0119H

Under the supervision of

PROF. TATHAGATA RAY

SUBMITTED IN PARTIAL FULLFILLMENT OF THE REQUIREMENTS OF

CS F266: STUDY PROJECT

BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI (RAJASTHAN)

HYDERABAD CAMPUS

(APRIL 2026)

ACKNOWLEDGMENT

At the very onset of this report, I would like to take the opportunity to profoundly thank Prof. Tathagata Ray for guiding me through the course of this project. Without his unwavering support, expert insights and constant feedback, this project would not have been possible.

I would also like to express my sincere gratitude to Ms. Pothuri Sowmya Sudha ma'am for her invaluable guidance and encouragement, which helped me confidently move forward through the various stages of this project.

Birla Institute of Technology and Science-Pilani,

Hyderabad Campus

Certificate

This is to certify that the project report entitled "**IMPLEMENTATION OF NEURAL NETWORK MESH GENERATION ALGORITHMS FOR 3D DOMAIN**" submitted by Mr. Aryan Agarwal (ID No. 2024A7PS0119H) in partial fulfillment of the requirements of the course CS F266, Study Project Course, embodies the work done by him under my supervision and guidance.

Date: April, 2026

(PROF. TATHAGATA RAY)

BITS- Pilani, Hyderabad Campus

Abstract

This project aims to design an algorithm to generate an unstructured mesh on an input contour shape. The algorithm aims to use a neural network-based approach for this task, with multiple models being used in a pipeline to accomplish the successive stages of predicting the number of vertices required, the location of these vertices and the connectivity matrix of these vertices.

The first phase of the project involved studying about what meshes are, exploring the problem at hand and understanding the need for an automated algorithm for creating meshes, over traditional methods of mesh generation. We also researched existing mesh generation models, to understand the progress that has already been made by the community in this domain, by reading through the literature that has been written on this subject, including blogs, journals and most importantly, research papers.

The next step was to start implementing some of the algorithms that these papers detailed, and the algorithm chosen for this project was the one by Sumedh Soman and Ninad Mehendale, in their paper titled 'Faster and efficient tetrahedral mesh generation using generator neural networks for 2D and 3D geometries'.

This report highlights the insights gained from the research, as well as the results of implementing and testing the aforementioned algorithm on various shapes. Finally, it further presents the future steps that can be taken in this project.

Contents

Chapter 1

Introduction to Neural Network based Algorithms

Mesh generation, which is the process of partitioning a continuous geometric domain into discrete elements, has become an indispensable part of computational science. Whether it be simulating airflow over a wing or visualizing blood flow through an artery, the accuracy of the result of such experiments is inextricably linked to the quality of the mesh used, and this quality is inherently decided by the quality of the underlying algorithm used to generate the mesh.

Traditionally, mesh generation represented a rigorous mathematical task, and was accomplished by approximating solutions to a series of partial differential equations on the grid, most commonly Poisson's equations.

These methods, though reliable in creating meshes of decent quality, are computationally very expensive, and not feasible as the application scales, or for complex contours. Thus, there arose a need for an automated process to construct such meshes using a different method, and the field underwent a paradigm shift toward neural networks, leveraging deep learning to overcome the scalability and complexity limits of traditional methods. This shift stemmed from the realization

Moving from solving PDEs to using neural networks represents a massive change in how the mesh generation problem is approached, and drastically reduces the time and computational power required to mesh a given shape. Neural networks, once they have been trained on sufficient data, can instantly generate meshes of comparable quality to the best Delaunay mesh generators, using internal parameters learned from the training data.

Mesh generation using neural networks have several advantages over the traditional mathematical methods:

- **Speed:** Since neural network based mesh generation does not actually require performing complicated maths such as approximation of solutions to PDEs, it is much faster than all the other mesh generation algorithms discussed above. Furthermore, since neural networks form the mesh in one-pass, rather than iteratively, this further reduces the time required.
- **Efficiency:** Neural networks are much better at refining and altering a mesh, once generated, and does not need to regenerate an entire mesh for small alterations to

the input contour, making it highly useful in applications where the object to be meshed undergoes slight changes in shape and size often.

- **Adaptability:** Once trained on a set of reference shapes, a neural network based algorithm can mesh a new shape in no time, and does not need to relearn on more reference data.
- **Versatility:** Traditional mesh generation struggles when the domain is complex or has imperfections. However, neural network based generators can effectively learn to handle such issues and fix the geometry before meshing.

However, there are also some issues that neural network mesh generation methods entail. These can be enumerated as follows:

- **'Black Box' Risk:** Unlike traditional Delaunay methods, neural networks do not guarantee mathematical accuracy and might misplace a vertex, leading to overlapping triangles or tangled meshes that will reduce the mesh quality.
- **Data Dependency and Training Cost:** Neural networks require a massive amount of data to train, and this might not be feasible for many applications where such amounts of training data is not available. Furthermore, training the model on large datasets also represents a significant computational cost.
- **Generalization:** While traditional solvers can usually adapt to any contour or shape by adjusting the PDEs accordingly, neural network models that have been primarily trained on a certain class of contours might not be able to generalize to unseen shapes or domains without additional training.

These problems, and others, represent the subject of ongoing research in this field, and newer algorithms are continuously being developed to tackle one or more of these problems. Areas of research include operator learning, graph neural networks (GNNs), hybrid solvers using PINNs (Physics-informed Neural Networks) and more.

We analysed one such algorithm published in a research paper by Soman et al. The analysis is presented in the following section.

Analysis of Soman et al. Paper

This section presents the analysis of the paper titled 'Faster and efficient tetrahedral mesh generation using generator neural networks for 2D and 3D geometries' by Sumedh Soman and Ninad Mehendale, published in *Neural Computing and Applications* (2024).

This paper achieved a significant breakthrough in the neural network mesh generation landscape by introducing a Conditional Generative Adversarial Network (CGAN) based mesh generator, which achieved much faster rates as compared to conventional solvers.

The CGAN architecture operates by taking two specific inputs: the convex hull (representing the geometric boundary of the domain) and the target edge length for the mesh. The generator is trained to predict internal node coordinates, learning to replicate the high-quality node distributions found in training examples.

For training the neural network, the paper uses DistMesh, a well-known mesh generation algorithm based on force-displacement principles, with an Adam optimizer. The algorithm iteratively adjusts the positions of the nodes until the edges reach an equilibrium that produces high-quality meshes with the desired edge length. By using these meshes as the base-level training data, these meshes enable the network to map geometric features to high-quality node distributions.

Once the node positions have been finalized, a Delaunay triangulation algorithm is applied to connect the nodes and construct the final mesh elements.

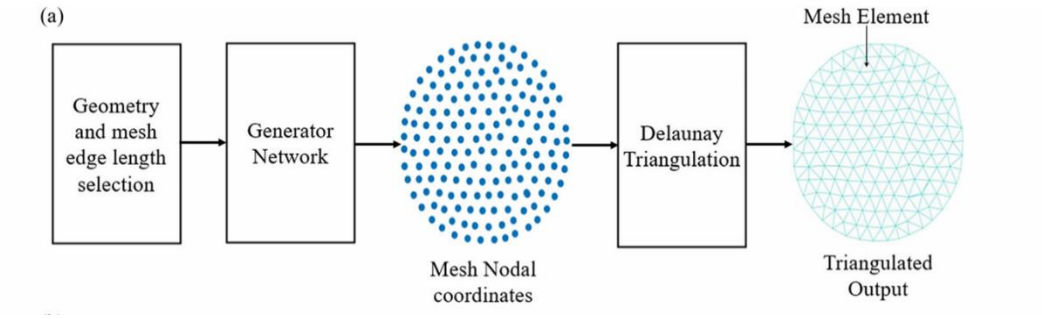


Fig 1.1 – Complete pipeline of the mesh generation algorithm

This algorithm, therefore, combines the advantages of machine learning to efficiently determine the placement of mesh nodes, and classical computational geometry ensures a valid mesh topology and prevents intersecting elements.

During training, the neural network is optimized by minimizing errors using an error loss function that measures the difference between the predicted node coordinates and the node coordinates obtained from the reference meshes generated using DistMesh.

The expression for the loss function is

$$J = \frac{-1}{N} \sum_{j} m_j w_j (Y_j - T_j)^2$$

Where:

- N = total number of training samples
- m_j = masking parameter used to indicate whether a node contributes to the loss
- w_j = weighting factor for the j^{th} node
- Y_j = predicted output from the neural network
- T_j = target (reference) value from the training mesh dataset

This loss function calculates the weighted squared error between the predicted and reference node positions. The weights allow certain nodes to contribute more strongly, thereby increasing the overall accuracy of the generated mesh.

Overall, the paper achieved many advancements over previous algorithms, including the following:

- 75% faster than DistMesh, and 13x times faster than ANSYS Mechanical Meshing.
- <1% difference in quality between modeled meshes and training meshes.
- Capable of meshing for both 2D and 3D domains.

Chapter 2

Implementation of Soman et al. Paper

Once we had properly studied and understood the algorithm proposed in the Soman et al. paper, we began working on actually implementing and coding the algorithm ourselves. To get started initially, we referred to the code attached with the paper as a base, from which we obtained the important mathematical functions, and network architecture from.

The implementation was divided into multiple phases, representing the entire pipeline of building the algorithm, from creating the training dataset to building the neural network to evaluating the model to compare with the metrics achieved by the paper.

The various phases are discussed in the following sections.

Phase 1 – Generation of Reference Meshes

The first step, before we built any models, was to create the reference meshes using existing libraries for traditional Delaunay-based algorithms.

While the paper used the DistMesh algorithm, for the purposes of this project, we used the TetGen library, a Delaunay-based mesh generator developed by Hang Si. This library is widely used to create high quality tetrahedral meshes for 3D shapes, using the CDT (Constrained Delaunay Triangulation) method.

We started off by testing the library to generate a simple sphere. Then we proceeded to create the actual reference mesh dataset. Our dataset contained five reference meshes of varying minratios for each of 3 geometries; sphere, cylinder, and torus. Further, we varied the respective resolution parameters for each shape to create test meshes of varying densities. For each shape, we first used PyVista to create the triangulated surfaces, and then TetGen to tetrahedralize the internal region before extracting and storing the nodes. In total, we collected 15 sphere samples, and 10 cylinder and torus samples each.

```
Geometry: sphere (label=0)
  res=[theta_resolution=20, phi_resolution=20], minratio=1.3 →  1048 nodes
  res=[theta_resolution=20, phi_resolution=20], minratio=1.5 →  1098 nodes
  res=[theta_resolution=20, phi_resolution=20], minratio=1.8 →  1158 nodes
  res=[theta_resolution=20, phi_resolution=20], minratio=2.0 →  1170 nodes
  res=[theta_resolution=20, phi_resolution=20], minratio=2.5 →  1387 nodes
  res=[theta_resolution=25, phi_resolution=25], minratio=1.3 →  1584 nodes
  res=[theta_resolution=25, phi_resolution=25], minratio=1.5 →  1582 nodes
  res=[theta_resolution=25, phi_resolution=25], minratio=1.8 →  2121 nodes
  res=[theta_resolution=25, phi_resolution=25], minratio=2.0 →  2239 nodes
  res=[theta_resolution=25, phi_resolution=25], minratio=2.5 →  2606 nodes
  res=[theta_resolution=30, phi_resolution=30], minratio=1.3 →  2485 nodes
  res=[theta_resolution=30, phi_resolution=30], minratio=1.5 →  2789 nodes
  res=[theta_resolution=30, phi_resolution=30], minratio=1.8 →  3564 nodes
  res=[theta_resolution=30, phi_resolution=30], minratio=2.0 →  4133 nodes
  res=[theta_resolution=30, phi_resolution=30], minratio=2.5 →  4301 nodes
```

(a)

```
Geometry: cylinder (label=1)
  res=[resolution=30], minratio=1.3 →  1377 nodes
  res=[resolution=30], minratio=1.5 →  1408 nodes
  res=[resolution=30], minratio=1.8 →  1366 nodes
  res=[resolution=30], minratio=2.0 →  1493 nodes
  res=[resolution=30], minratio=2.5 →  1585 nodes
  res=[resolution=40], minratio=1.3 →  2092 nodes
  res=[resolution=40], minratio=1.5 →  1961 nodes
  res=[resolution=40], minratio=1.8 →  2191 nodes
  res=[resolution=40], minratio=2.0 →  2076 nodes
  res=[resolution=40], minratio=2.5 →  2683 nodes
```

(b)

```
Geometry: torus (label=2)
  res=[u_res=20, v_res=20, w_res=20], minratio=1.3 →  1671 nodes
  res=[u_res=20, v_res=20, w_res=20], minratio=1.5 →  1584 nodes
  res=[u_res=20, v_res=20, w_res=20], minratio=1.8 →  1903 nodes
  res=[u_res=20, v_res=20, w_res=20], minratio=2.0 →  2103 nodes
  res=[u_res=20, v_res=20, w_res=20], minratio=2.5 →  2722 nodes
  res=[u_res=25, v_res=25, w_res=25], minratio=1.3 →  2823 nodes
  res=[u_res=25, v_res=25, w_res=25], minratio=1.5 →  3078 nodes
  res=[u_res=25, v_res=25, w_res=25], minratio=1.8 →  3495 nodes
  res=[u_res=25, v_res=25, w_res=25], minratio=2.0 →  3714 nodes
  res=[u_res=25, v_res=25, w_res=25], minratio=2.5 →  4197 nodes
```

(c)

Fig 2.1 – Reference meshes created by TetGen for three geometries;
(a) Sphere, (b) Cylinder, (c) Torus

Phase 2 – Data Preparation

The next step was to prepare the reference data for the model. This included scaling down and normalizing the data, then packing it into a fixed-size input matrix. Both of these steps were essential to ensure that the parameters of training mesh data matched the output generated by the CGAN network.

The normalization was done using simple min-max scaling to bring the node coordinates within the [-1, 1] range. This was required as the network's output layer uses the tanh function which outputs in this range. Each mesh was normalized using its respective bounding box.

Then, we had to pack the coordinates into a fixed size tensor with 3 channels for x,y,z coordinates each. This was done using basic NumPy array operations. The fixed size was chosen as the smallest power of 2 greater than the maximum node count among all reference meshes. For each mesh, we padded the empty space with zeroes, and masked these to prevent the network from using these points, which would all lie at the origin (0,0,0).

The last step was to wrap the list of reference meshes in a standard PyTorch dataset, to allow us to use the DataLoader to batch, shuffle and feed the data to the network automatically during training.

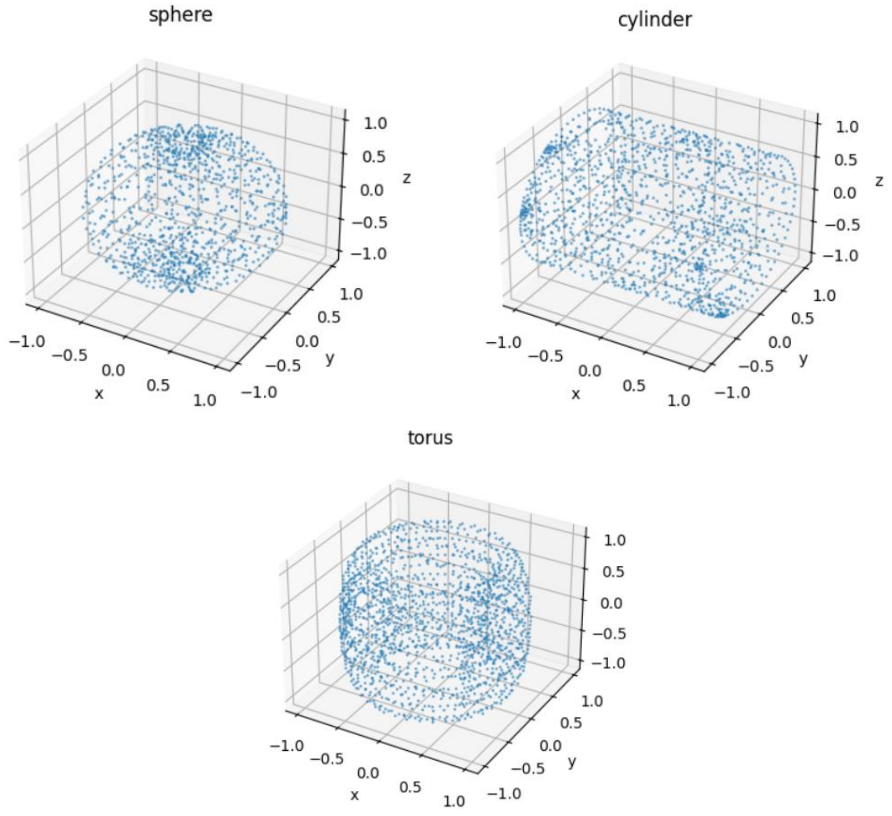


Fig 2.2 – Node distributions for a sample surface from each geometry

Phase 3 – Building the CGAN

Now, we were finally ready to build and train the actual CGAN network. The process of building the network involved multiple steps, enumerated below:

- Defining network hyperparameters
- Label embedding
- Building the generator block
- Assembling full CGAN
- Verification

The first step in this phase was to define the CGAN's hyperparameters. This was done to ensure that the dimensions defined in the previous phase for the training data match the output matrix dimensions generated by the network. Further, we also defined other parameters for the deconv layers such starting spatial size and number of filters, as well as the size of the embedding layer for the next step.

Next, we created the label embedding block, or the 'conditional' part of the CGAN. This part of the architecture embeds the labels for each geometry, which were simply assigned as integers 0, 1 and 2; the output generated is dense vector representation of the geometry. This embedding block consists of a sequence of three layers:

- Embedding Layer – Maps integer labels to a learned vector of fixed size defined in the previous step.
- Linear Layer – Reshapes the embedding into a flat representation.
- ReLU Layer – Captures non-linear patterns within the geometry that might be missed in the linear layer.

After this, we implemented a single generator block. The function of a single block was to upsample the nodes and double the dimensions of the input channel by using a 2D convolutional layer with a stride of 2. The entire block consists of a sequence of three layers:

- ConvTranspose2d Layer – Creates a kernel that upsamples the input 2D feature map to double the spatial size.
- BatchNorm2d – Performs batch normalization on the activation in each batch, stabilizing training and speeding up convergence.
- ReLU Layer – Used for hidden layers to introduce non-linearity.

Finally, with all the individual modules ready, we were now in a position to design and build the complete CGAN neural network. After testing a few different architectures and layer structures, the final version we settled with was the following one:

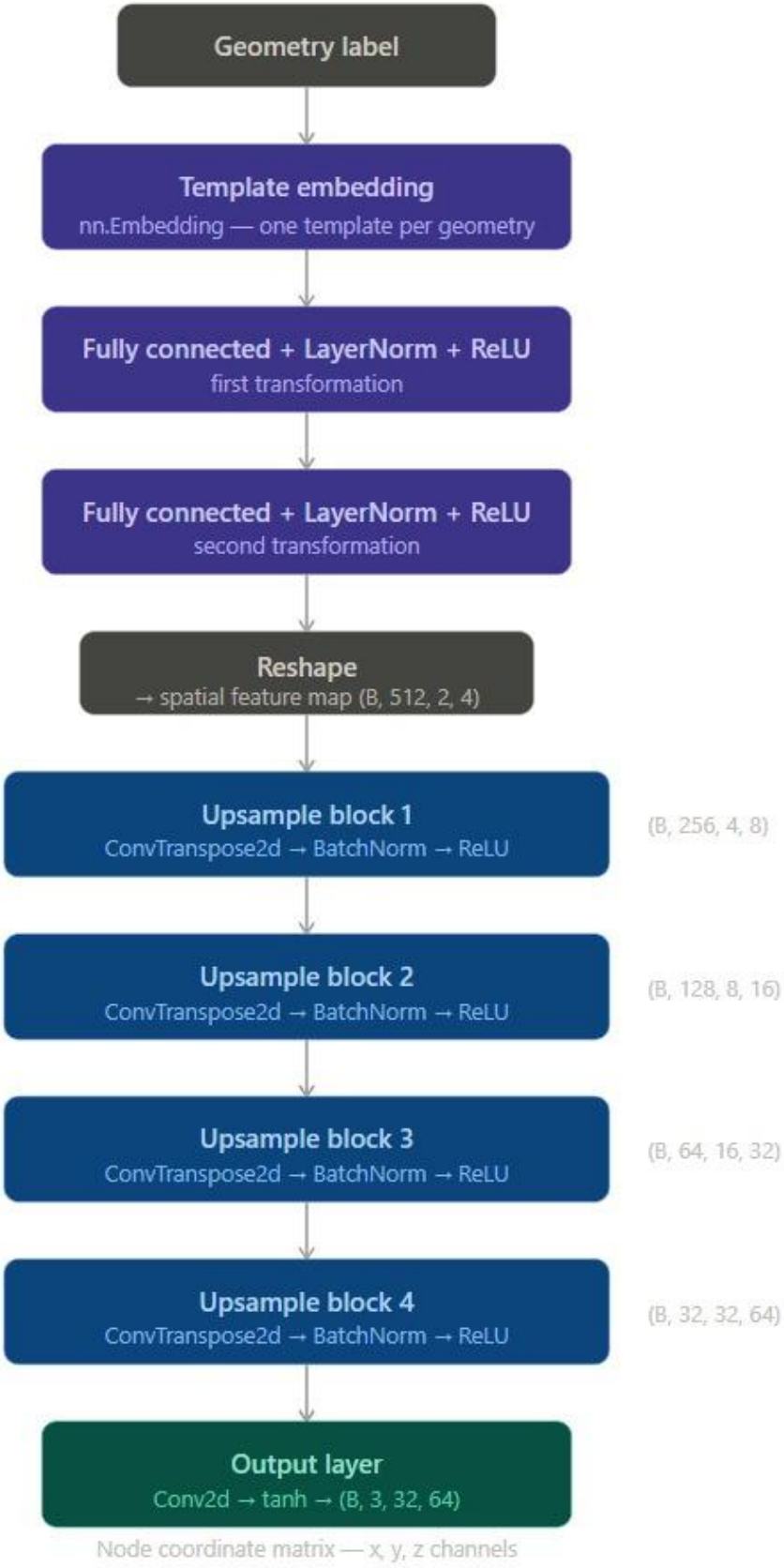


Fig 2.3 – CGAN Architecture

As can be seen in the above image, the layer breakdown is as follows:

- Embedding Layer – To create label embeddings for multiple geometries.
- Fully-connected (FC) Dense Layers – Transform and learn each template before upsampling.
    - Linear Layer
    - LayerNorm Layer
    - ReLU Layer
    - Linear Layer
    - LayerNorm Layer
    - ReLU Layer
- Upsampling Layers – Increases spatial size of input map four times to allow generator to capture finer details.
    - GeneratorBlock Layer (x4)
- Output Layer – Sizes feature map back down to original size, scales it and outputs final result.
    - Conv2d Layer
    - Tanh Layer

Once the architecture was completely built, we also did a quick verification by performing a singular forward pass with dummy inputs to eliminate any accidental shape mismatches in any layer early on.

Phase 4 – Training the CGAN

Once the final generator architecture was ready, we started training it on the reference mesh data. The training process followed the following pipeline:

- Defining training hyperparameters
- Creating optimizer and scheduler
- Defining MSE loss function
- Training loop for CGAN with logging and convergence checkers
- Plotting loss curve
- Saving best weights

The first step in training was defining the training hyperparameters for the model, such as the learning rate, epoch count, stop loss and Adam optimizer betas.

Next, we created the Adam optimizer using the hyperparameter values defined earlier, as well as the scheduler, whose job is to change the learning rate parameter of the optimizer if loss doesn't improve after a certain number of epochs.

Then, we defined the masked MSE (mean squared error) loss function that would be used to calculate the loss after each epoch. The masking was used here as well to ensure that the loss was only calculated for real node positions, and that the gradient would not be driven by empty padding nodes.

With all the tools in hand, we then ran the actual training loop for the CGAN. For each epoch, the following events took place:

- First, the matrices, masks and labels were loaded in from the PyTorch DataLoader.
- For each geometry, the forward pass was made on the data to get predictions of nodal coordinates for the associated label.
- Next, the masked loss for that geometry was computed.
- Then the backward pass for that geometry was performed, and weights were updated.
- Average loss among all three geometries for the epoch was calculated and stored.

Furthermore, every 50 epochs, a progress update was printed. Every epoch was also checked for convergence in-case the loss reached the stop loss value. The best epoch had its weights and loss value stored, which were printed at the end of the loop.

Once the training loop completed, we plotted the loss curve to see how the model progressed over all epochs. The results were as shown:

```
Starting training...
  Samples        : 35
  Batch size     : 3
  Batches/epoch  : 12
  Max epochs     : 500
  Stop loss      : 0.02
  Device         : cuda
```

(a)

(b)

Fig 2.4 – (a) Details of the training process, (b) Log loss curve for 500 epochs

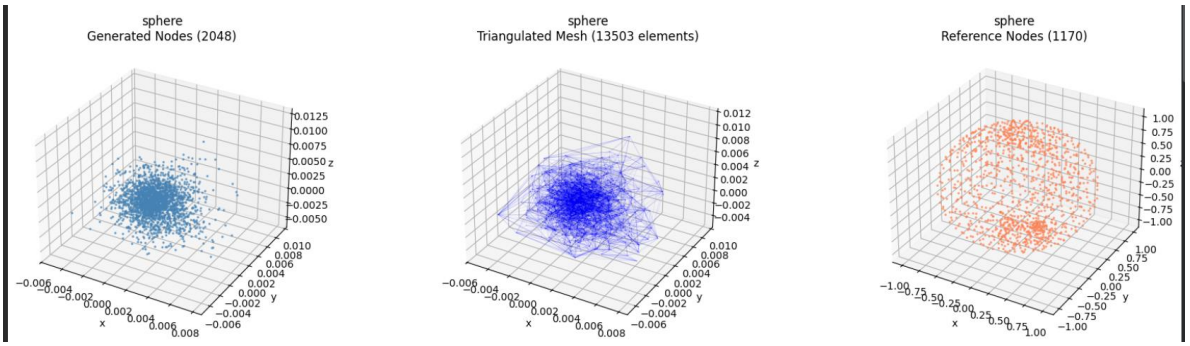
The best loss achieved was 0.17. With the details of the corresponding epoch stored, it was finally time to move to the final phase of the implementation.

Phase 5 – Evaluating the Results

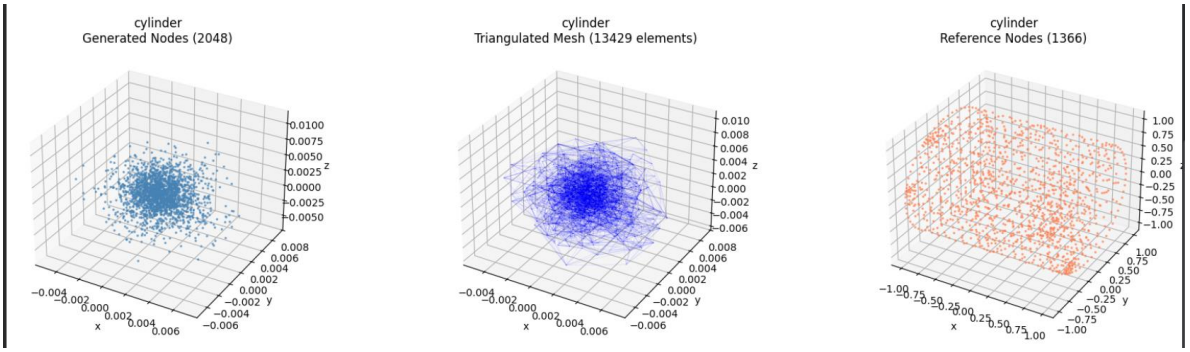
After training the network, the last thing left to do was unpack and interpret the output matrix of nodes, mesh it using Delaunay methods, visualize the results, and evaluate the resulting mesh on external metrics as well as metrics defined in the paper.

The first step was to convert the output matrix back into a series of node coordinates, making sure only real nodes are unpacked and not masked padding, by using a threshold filter for the distance of the node from the origin (0,0,0).

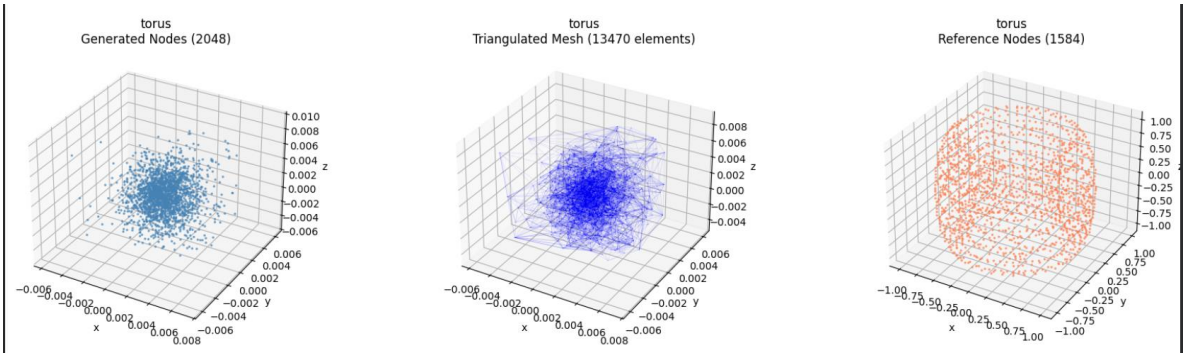
Next, we used SciPy's Delaunay triangulation function over the nodes to create the connectivity matrix and form the actual mesh elements.



(a)



(b)



(c)

Fig 2.5 – Visualization of internal Steiner points, mesh edges and surface node distribution for (a) Sphere, (b) Cylinder, (c) Torus

The next step was to visualize the mesh generated by the network, including the Steiner points, as well as edge connectivity, as is seen in the above image.

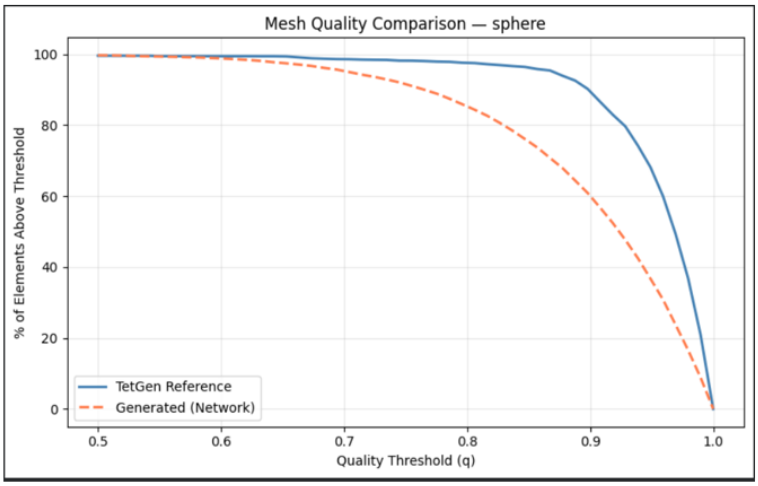
After that, we then evaluated and compared the TetGen and generated meshes on the field quality metric, as was mentioned in the paper. This metric was calculated for every single element in the mesh. The equation used was:

$$q = \frac{(b+c-a)(c+a-b)(a+b-c)}{abc}$$

A value of q = 1 denotes an ideal element, with values up to q >= 0.5 also considered acceptable. We calculated the percentage of elements with q >= 0.9, q >= 0.5, and q < 0.5. Also, we graphed the cumulative quality distribution curve.

```
Quality Report — sphere (TetGen):
  Total elements : 5794
  Mean quality   : 0.952
  q >= 0.9       : 89.7%
  q >= 0.5       : 99.6%
  q <  0.5       : 0.4%  ← poor elements
```

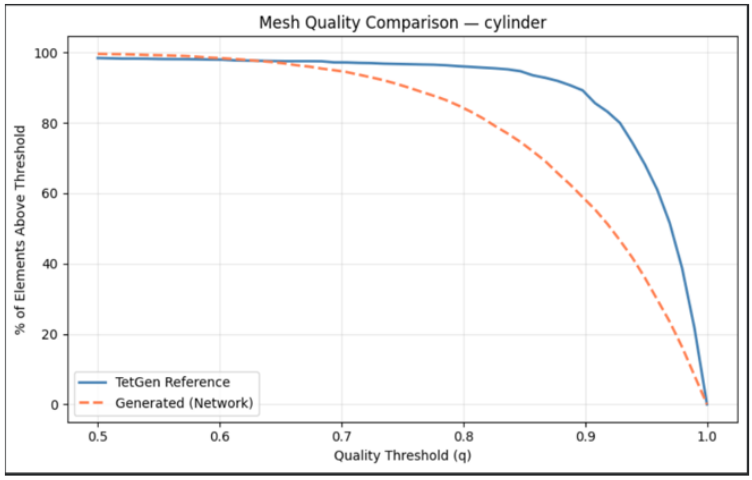
```
Quality Report — sphere (Generated):
  Total elements : 13498
  Mean quality   : 0.896
  q >= 0.9       : 60.1%
  q >= 0.5       : 99.7%
  q <  0.5       : 0.3%  ← poor elements
```



(a)

```
Quality Report — cylinder (TetGen):
  Total elements : 6322
  Mean quality   : 0.944
  q >= 0.9       : 88.9%
  q >= 0.5       : 98.4%
  q <  0.5       : 1.6%  ← poor elements
```

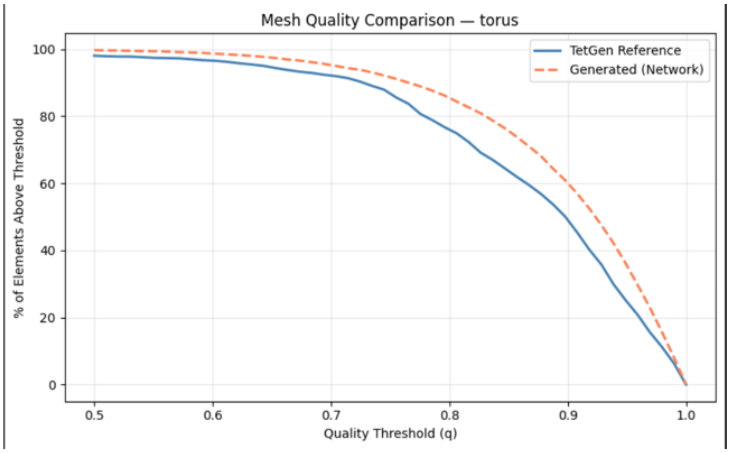
```
Quality Report — cylinder (Generated):
  Total elements : 13424
  Mean quality   : 0.892
  q >= 0.9       : 58.2%
  q >= 0.5       : 99.6%
  q <  0.5       : 0.4%  ← poor elements
```



(b)

```
Quality Report — torus (TetGen):
  Total elements : 9367
  Mean quality   : 0.863
  q >= 0.9       : 49.2%
  q >= 0.5       : 98.0%
  q <  0.5       : 2.0%  ← poor elements
```

```
Quality Report — torus (Generated):
  Total elements : 13469
  Mean quality   : 0.896
  q >= 0.9       : 60.1%
  q >= 0.5       : 99.7%
  q <  0.5       : 0.3%  ← poor elements
```



(c)

Fig 2.6 – Field quality metric scores for reference TetGen mesh vs generated mesh for (a) Sphere, (b) Cylinder, (c) Torus

The results for field quality metric scores in the TetGen meshes vs in the generated meshes, as the cumulative quality distribution curve, for each of the three geometries, is shown in the above image.

Key Insights and Takeaways

The results obtained from the implementation were very positive, and we were able to replicate the algorithm presented in the paper to a satisfactory extent.

The overall summary of the field quality metric results for the generated meshes was:

```
================================================
FINAL QUALITY SUMMARY
================================================
Geometry       Elements    Mean q   q>=0.9   q>=0.5
------------------------------------------------
sphere            13498     0.896    60.1%    99.7%
cylinder          13424     0.892    58.2%    99.6%
torus             13469     0.896    60.1%    99.7%
```

As seen above, the mean element quality consistently stayed around 0.9 for all geometries, close to excellent, just slightly shy of the paper's reported ~0.94 score. Furthermore, the difference in quality of elements in the generated vs TetGen meshes hovered well within the 1% range described in the paper.

Across geometries, 99.6-99.7% of elements had an acceptable quality metric of q >= 0.5, i.e. almost all elements were simulation-ready with very few unusable elements.

One standout result was that of the torus, in which the element quality of the generated mesh outperformed the TetGen mesh quality; with 60.1% vs 49.1% elements at q >= 0.9. This indicates that the network learned internal node spacing better than conventional mesher for the most complex geometry of the three!

Summary

In summary, in this project, we analyzed and implemented an entire pipeline for a mesh generation algorithm using a Conditional Generative Adversarial Network (CGAN) architecture, based off of an algorithm proposed in the research paper titled 'Faster and efficient tetrahedral mesh generation using generator neural networks for 2D and 3D geometries' by Sumedh Soman and Ninad Mehendale.

The pre-midsem phase of this project involved studying and exploring the concepts of meshes, triangulation and mesh generation algorithms, which included both conventional and neural network based methods. As part of this study, we also analysed two landmark research papers in this field, the aforementioned paper by Soman et al. and the paper titled 'How to teach neural networks to mesh: Application on 2-D simplicial contours' by Papagiannopoulos et al.

The post-midsem phase of this project focused on implementing the algorithm proposed in the Soman et al. paper, for which we created an entire pipeline. This consisted of 5 phases; generation of the reference meshes using TetGen, preparation and preprocessing of the data, building the CGAN architecture, training the network on the reference data, and finally evaluating the performance of the generator against TetGen using metrics defined in the paper. We also added features beyond what the paper presented, including template embeddings, masking in the loss function and an extra upsampling stage.

The findings of the project were then presented in a concise summary using the model scores and graphs, for easy visualization of the network output results. We were able to achieve comparable results to the paper, and the mesh quality was on par with the ones generated by a conventional mesher like TetGen.

Limitations and Next Steps

While this implementation of the Soman et al. algorithm was quite successful, there were certain limitations and areas with scope for improvement that surfaced over the course of this project.

One of the main areas where the generator built for this project lagged could not replicate the result presented in the paper was in minimizing the loss function. While the paper was able to achieve a minimum L^2 loss value of 0.0298, the implemented network could only achieve a minimum of ~0.17, which represents a sizeable gap in performance.

This gap can partly be attributed to the computing limitations we had. While the original authors trained their model with 50+ samples per geometry, using a generator + discriminator architecture for their CGAN, on a high-performance Nvidia RTX 2060 GPU; we could only train with 10-15 samples per geometry, using a generator only CGAN, on CPU. This disparity in computing power available caused the loss function to plateau at 0.17.

Another aspect of the paper that was not implemented in this project was support for 2D input shapes as well. However, 2D was not the main aim of this implementation and hence was not added, but can easily be included with some modifications to the code.

Lastly, we did not add functionality to time our generator to perform a speed-wise comparison against the paper's benchmark of being 75% faster than the DistMesh algorithm.

To address the above-mentioned issues, as well as to start improving the algorithm above the paper's benchmark, the future steps that can be taken are as follows:

- Train the model on a more powerful GPU with more training samples and with adversarial training included; in order to meet the loss value achieved in the paper.
- Explore more complex geometries to train the model on, to increase its generalization across a variety of arbitrary input shapes. For this, the internal label embedder can be replaced with a PointNet encoder to handle any shape. The implementation for this has already been started, and is actively being worked upon.
- Adding support for 2D and algorithm speed measurement to fully complete the implementation of the paper.

References

1. Soman, S. & Mehendale, N. (2024). Faster and efficient tetrahedral mesh generation using generator neural networks for 2D and 3D geometries. *Neural Computing and Applications (36),* 1805-1813. https://doi.org/10.1007/s00521-023-09119-2

2. Si, H. (2015). "TetGen, a Quality Tetrahedral Mesh Generator and a 3D Delaunay Triangulator." ACM Transactions on Mathematical Software (TOMS), 41(2), 1-36. https://doi.org/10.1145/2629697

3. Gao, W., et al. (2022). "TetGAN: A Convolutional Neural Network for Tetrahedral Mesh Generation." British Machine Vision Conference (BMVC). https://doi.org/10.48550/arXiv.2210.05735